Analysis of the MaxProductOfThree Algorithm (Non-Recursive)

1. Algorithm Description

The algorithm computes the maximum product of any three distinct elements in a non-empty zero-indexed array A consisting of n integers.

Instead of checking all possible triplets (which would be inefficient), the algorithm performs a **single linear pass over the array** while maintaining:

- The **3 largest** elements (max1 ≥ max2 ≥ max3)
- The **2 smallest** elements (min1 ≤ min2)

At the end, it calculates two candidate products and returns the larger one:

- Product of the three largest numbers
- Product of the two smallest numbers and the largest number

This approach efficiently handles both positive and negative numbers.

2.Input and Output

• Input: An array of integers of size n ≥ 3.

• Output: The maximum product of any three distinct elements as a single integer.

3. Input Size

The input size is defined as:

n = number of elements in array A, where **n ≥ 3**.

4. Basic Operation

The **basic operation** is the **comparison** operation (i.e., comparing the current element x with max1, max2, max3, min1, or min2).

5. Dependency on Input

The number of executions of the basic operation **depends only on n**, and **does not depend** on the specific values or arrangement of the elements in the array .

6. Time Efficiency Analysis

The algorithm consists of a single **FOR loop** that iterates over all n elements of the array exactly once.

Inside each iteration, a **constant number** of comparisons and assignments are performed (at most 5 comparisons).

*Using the **summation** notation for the **for loop***

Solving the summation step by step :

Let $\boldsymbol{t_i}$ be the number of times the basic operation is executed in the i-th iteration of the loop.

Then:

$$C(n) = \sum_{i=1}^{n} t_i$$

Since the loop runs exactly n times and each iteration performs at most 5 comparisons:

$$C(n) = \sum_{i=1}^{n} 5$$

$$\boldsymbol{C(n) = 5 \cdot n}$$

$$\boldsymbol{C(n) = 5n \in \Theta(n)}$$

Therefore, the running time T (n) is approximately:

$$T(n) \approx c_{op} \cdot C(n) = c_{op} \cdot \Theta(n) = \Theta(n)$$

Conclusion:

- Best-case time complexity: $\Theta(n)$
- Worst-case time complexity: $\Theta(n)$
- Average-case time complexity: $\Theta(n)$

Overall Time Complexity: Θ(n)

6. Space Complexity

The algorithm uses only five scalar variables (max1, max2, max3, min1, min2) regardless of the value of n. No additional data structures proportional to n are used.

Space Complexity = Θ(1)